COMM 401: Signal & System Theory

Audio Processing Project Report

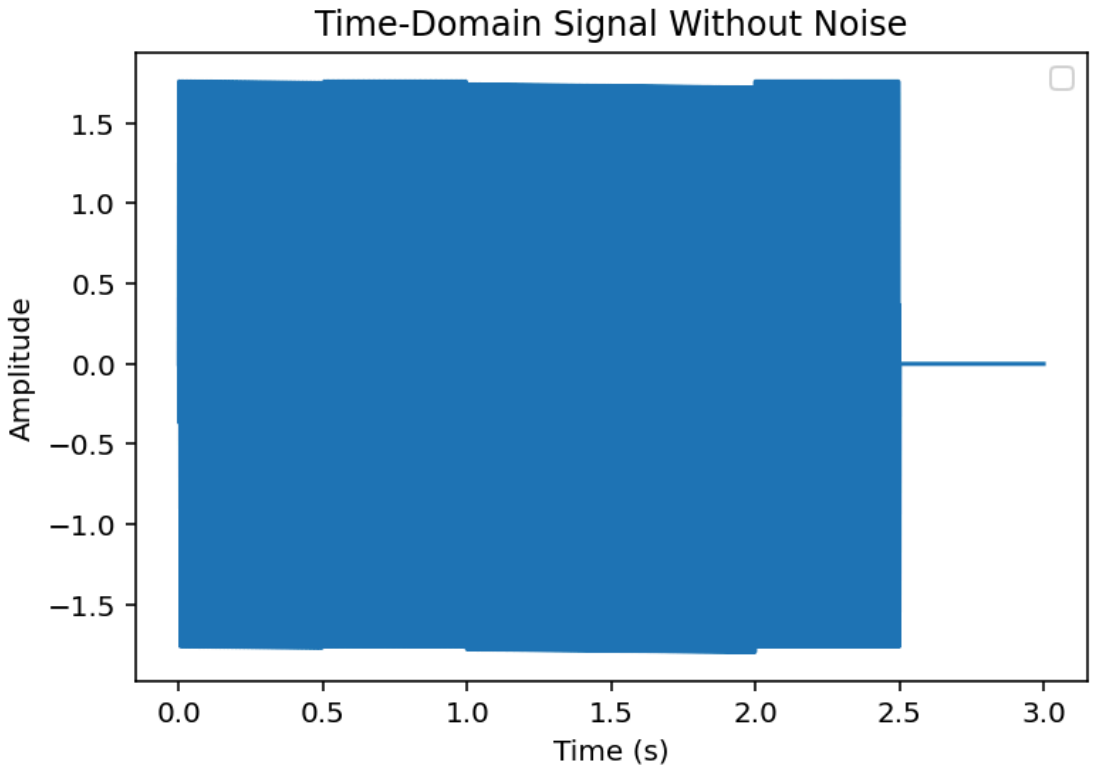
By:

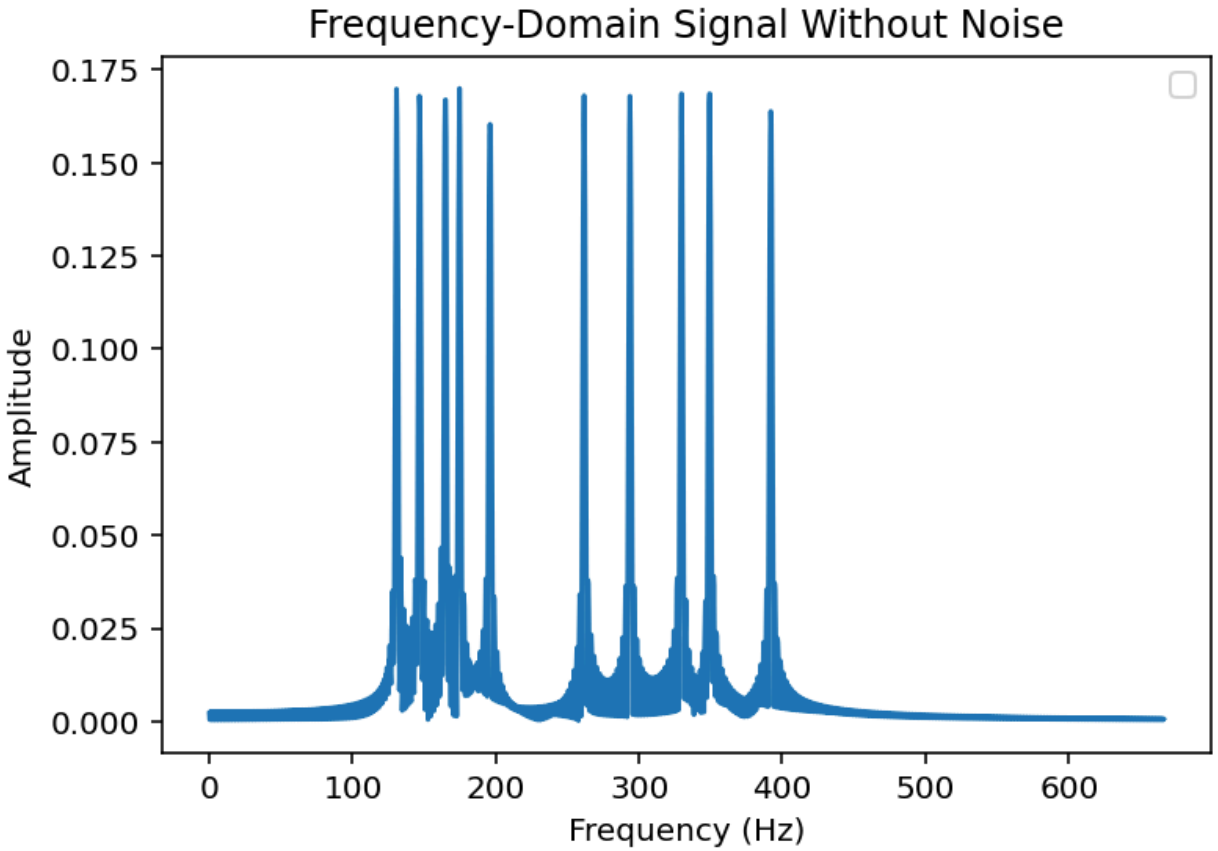
Salma Mohamed Omar, 61-3761
Omar Hatem, 61-34600

T.8, MET

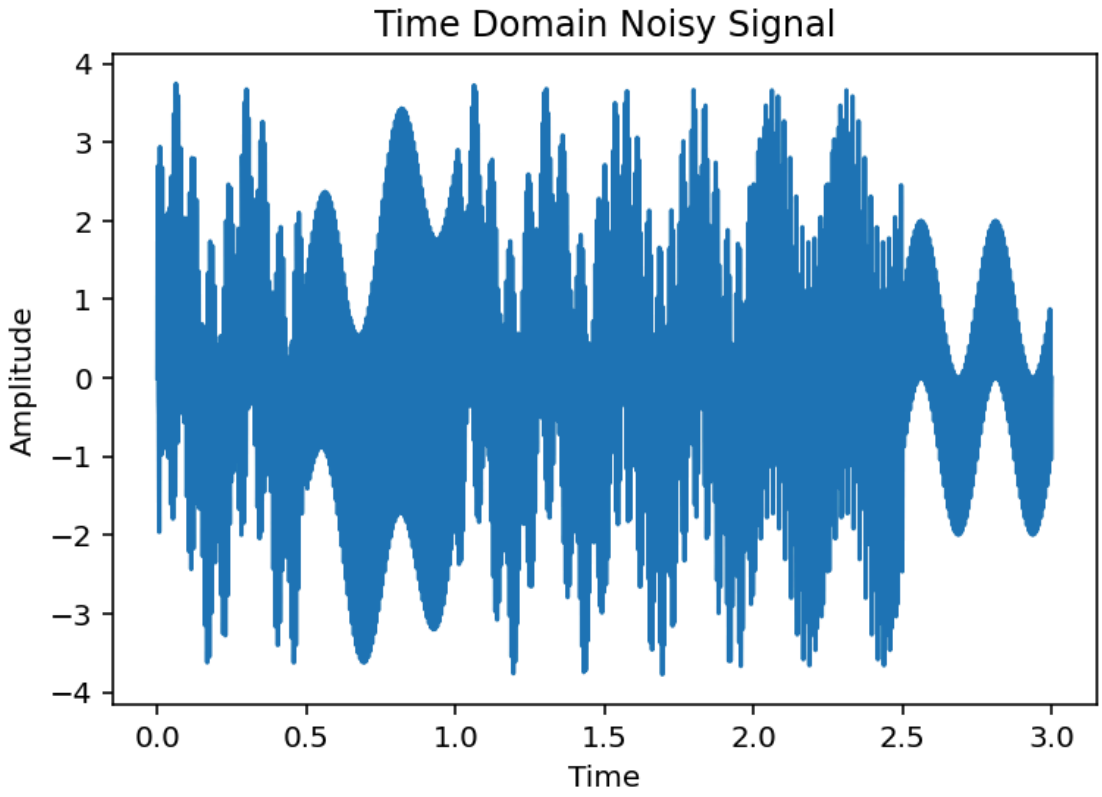
TA: Dr Youstina Megalli

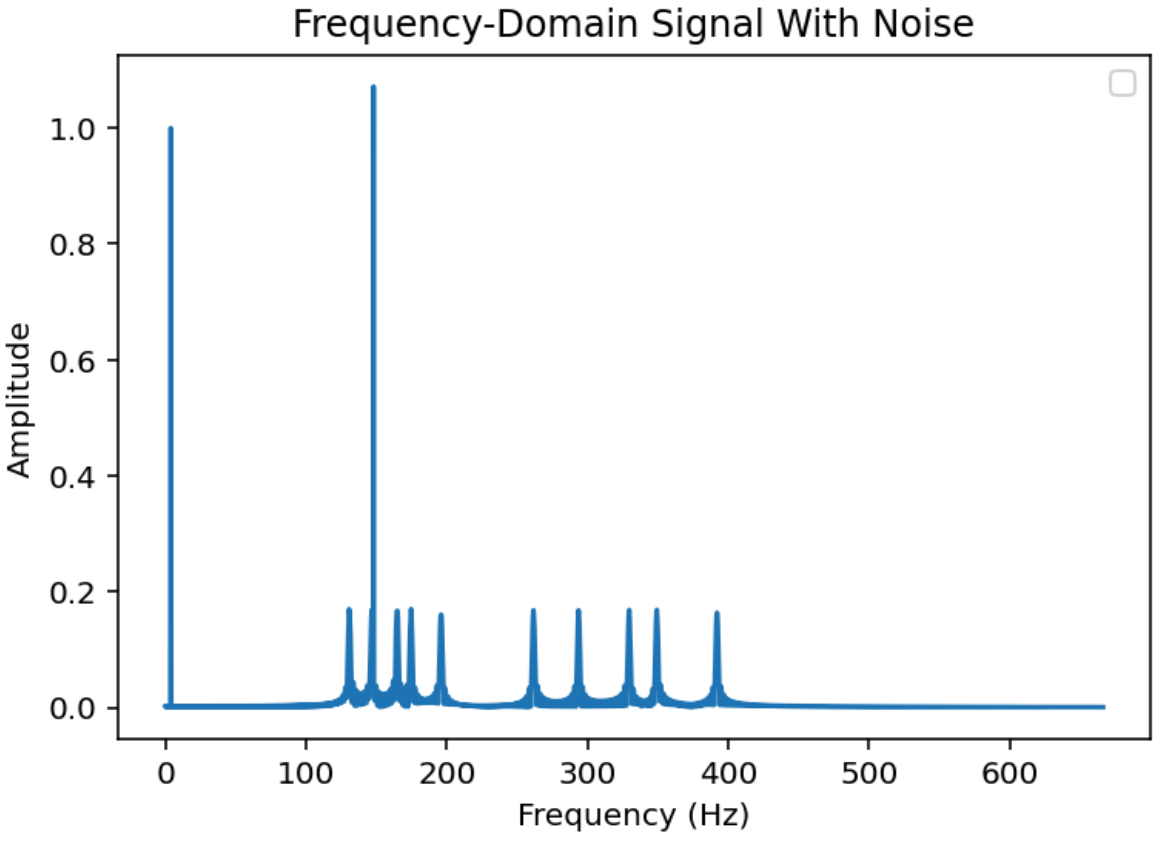
1- Signals Without Noise



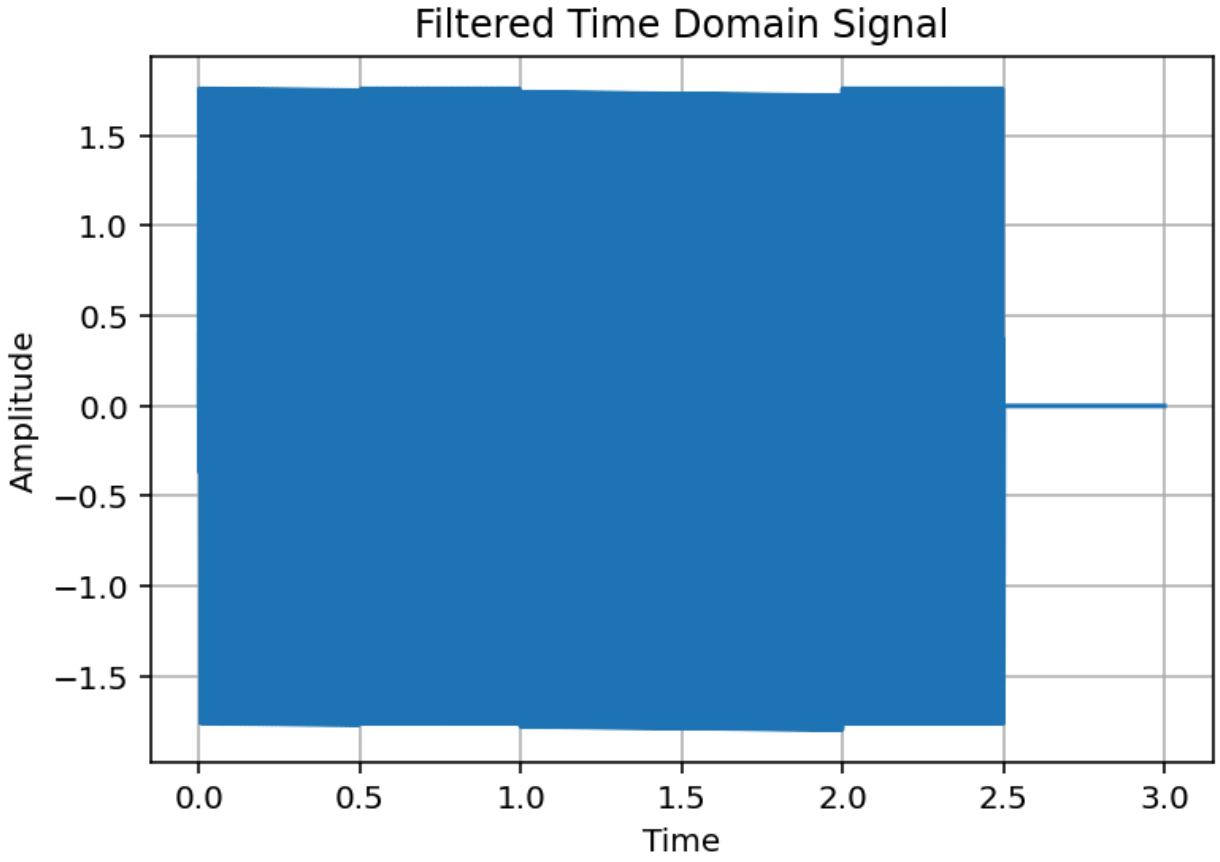


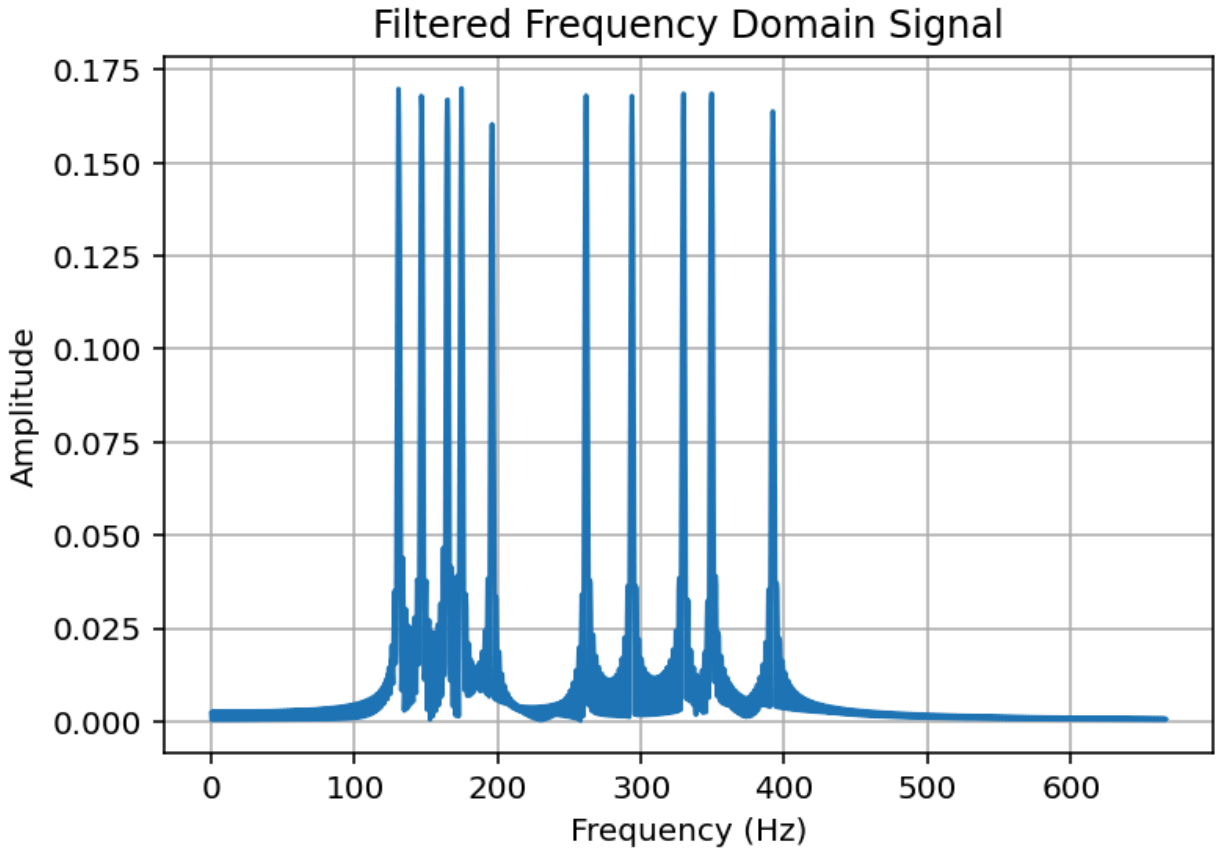
2- Signals With Noise





3- Filtered Signals





Python Script

```
# -*- coding: utf-8 -*-
"""
Created on Thu May  8 21:29:31 2025

@author: salmo
"""
import numpy as np
import matplotlib.pyplot as plt
import sounddevice as sd
from scipy.fftpack import fft
t = np.linspace(0, 3, 3 * 44100 )
N= 5

Fs = 44100
T_total = 3                # Total duration in seconds
#C = T_total * Fs
x = np.zeros_like(t)
octave3 = [130.81, 146.83, 164.81, 174.61, 196.00]
octave4 = [261.63, 293.66, 329.63, 349.23, 392.00]



start = [0, 0.5, 1.0, 1.5, 2.0]  # ti values (in seconds)
durations = [0.5, 0.5, 0.5, 0.5, 0.5]  # Ti values
i =0
while i < N :
    Fi = octave3[i]        # Note from 3rd octave
    fi = octave4[i]        # Note from 4th octave
    ti = start[i]    # Start time
    Ti = durations[i]      # Duration


    unit_step = np.logical_and(t >= ti, t < ti + Ti)


    x += (np.sin(2 * np.pi * Fi * t) + np.sin(2 * np.pi * fi * t)) *  unit_step
    i+=1


plt.plot(t, x)  # Plot part of the signal
```

```
plt.grid()
plt.title("Time-Domain Signal Without Noise")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid()
plt.legend(loc='best')
plt.show()



#frequency domain without noise
N = 3 * 44100
f=np.linspace(0,44100/2,int(N/2))
x_f = fft(x)
x_f = 2/N * np.abs(x_f[0:int(N/2)])
plt.plot(f[:2000],x_f[:2000])
plt.grid()
plt.title("Frequency-Domain Signal Without Noise")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.grid()
plt.legend(loc='best')
plt.show()

sd.play(x, Fs)
sd.wait()

#noise generation
fn1, fn2 = np.random.randint(0, 512, 2)
B= 44100
N = 2*B #Nyquist Rate
noise = np.sin(2*np.pi*fn1*t) + np.sin(2*np.pi*fn2*t)
noisy_signal = x + noise

plt.plot(t,noisy_signal)
plt.title ('Time Domain Noisy Signal')
plt.xlabel ('Time')
plt.ylabel ('Amplitude')
plt.show ()
```

```
#noisy signal in frequency domain
N = 3 * 44100
f=np.linspace(0,44100/2,int(N/2))
n_f = fft(noisy_signal)
n_f= 2/N * np.abs(n_f[0:int(N/2)])
mag = 2/N * np.abs(n_f[0:int(N/2)])
plt.plot(f[:2000],n_f[:2000])

plt.grid()
plt.title("Frequency-Domain Signal With Noise")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.grid()
plt.legend(loc='best')
plt.show()
sd.play(noisy_signal,Fs)
sd.wait()

#noise filtering
from scipy.signal import find_peaks
peaks, _ = find_peaks(n_f)
sorted_peaks = sorted(peaks, key=lambda p: n_f[p], reverse=True)
peak1, peak2 = sorted_peaks[:2] #largest 2 peaks
for i in range(0,np.size(n_f)):
    if n_f[i]==peak1:
        fn1 = i
        if n_f[i]==peak2:
            fn2 = i

noise_to_be_removed = np.sin(2 * np.pi * fn1 * t) + np.sin(2 * np.pi * fn2 * t)
x_filtered = noisy_signal - noise_to_be_removed

# Plot filtered time domain
plt.plot(t, x_filtered)
plt.title('Filtered Time Domain Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
```

```
# Plot filtered frequency domain
x_filtered_fft = fft(x_filtered)
x_filtered_mag = 2 / N * np.abs(x_filtered_fft[:int(N / 2)])

plt.plot(f[:2000], x_filtered_mag[:2000])
plt.title('Filtered Frequency Domain Signal')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')
plt.grid()
plt.show()
sd.play(x_filtered, Fs)
sd.wait()
```